SOEN 6011 — Delivery 2

Function F7: x^y

Yichen Huang
Student ID: 40167688

Concordia University

July 26, 2026

Outline

1. Problem 5: From-Scratch Implementation and GUI
2. Problem 6: GitHub Repository and Documentation
3. Problem 7: Updated Requirements

Problem 5 — From D1 to D2

Delivery 1

- Textual user interface
- Integer powers by exponentiation by squaring
- Non-integer powers using library logarithm and exponential functions
- Validation through built-in numerical utilities

Delivery 2

- Tkinter graphical user interface
- Integer powers still use exponentiation by squaring
- Natural logarithm implemented from scratch
- Exponential function implemented from scratch
- Custom validation and exception handling

Main Modification

The D1 hybrid algorithm was preserved, but all mathematical operations required for non-integer exponents were reimplemented without using Python power, logarithm, or exponential functions.

Problem 5 — From-Scratch Constraints

Allowed

- Input and output operations
- Basic arithmetic operations
- Comparisons and loops
- Tkinter for GUI design
- Built-in and custom exceptions

Not Used for Calculation

- `pow()` or `math.pow()`
- The `**` operator
- `math.log()` and `math.exp()`
- `math.isfinite()`
- External numerical libraries

Implementation Decision

The calculation layer was separated from the GUI and implemented with custom arithmetic algorithms for integer powers, natural logarithms, and exponential values.

Compliance Check

The submitted production code contains no built-in power, logarithm, or exponential operation.

Problem 5 — Program Architecture

GUI and Input Layer

- `PowerCalculatorApp`
- Reads base x and exponent y
- Handles Calculate, Clear, and Exit
- Displays results and status messages
- Uses `parse_number()`

Validation and Exceptions

- `is_finite_number()`
- `is_integer_value()`
- Domain and range checks
- Custom exception hierarchy

Calculation Layer

`calculate_power(base, exponent)`

⇓

Integer-valued exponent?

Yes	No
⇓	⇓
`power_by_squaring()`	`natural_log()` + `exponential()`

Numeric Protection

`_checked_multiply()` detects overflow and severe underflow.

Problem 5 — Integer Exponent Implementation

Exponentiation by Squaring

- Used when y is integer-valued
- Requires $O(\log |y|)$ iterations
- Uses arithmetic and loop control only
- Does not use `pow()` or `**`

Supported Cases

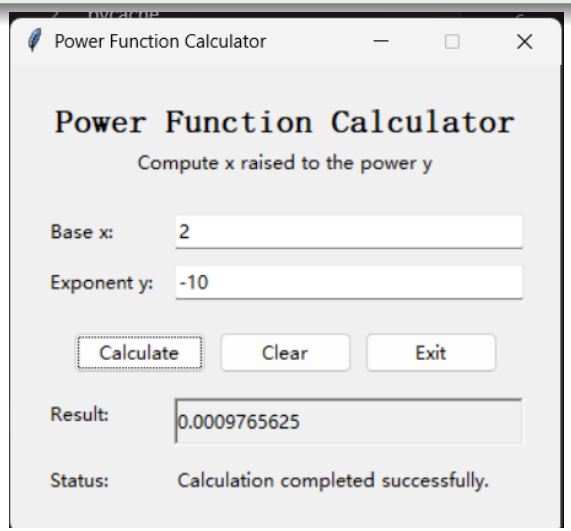
- $y = 0 \Rightarrow x^y = 1$
- $y > 0$: direct integer power
- $y < 0$: reciprocal of the positive power
- Negative bases are supported

Negative Exponent

For $y < 0$, the algorithm computes $x^{|y|}$ and returns its reciprocal.

Core Loop

```
result = 1
factor = base
while remaining > 0:
  if remaining is odd:
    result = checked multiply
  remaining = remaining // 2
  factor = factor * factor
```



GUI verification: $2^{-10} = 0.0009765625$

Problem 5 — Non-Integer Exponent Implementation

Hybrid Evaluation

For $x > 0$ and non-integer y:

$$x^y = e^{y \ln(x)}$$

Both $\ln(x)$ and e^t are implemented without Python mathematical library functions.

Custom Natural Logarithm

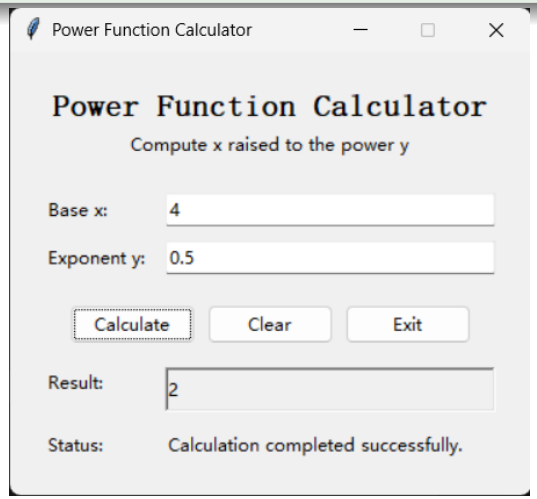
- Scales the input into $0.75 \leq m \leq 1.5$
- Uses $x = m \cdot 2^k$
- Computes

$$\ln(x) = \ln(m) + k \ln(2)$$

- Approximates $\ln(m)$ using an iterative series

Custom Exponential

- Writes $t = k \ln(2) + r$
- Approximates e^r with a Taylor series
- Updates each term iteratively
- Restores the scale using 2^k



GUI verification: $4^{0.5} = 2$

Convergence Control

A tolerance of 10^{-16} and a maximum of 1000 iterations prevent uncontrolled loops.

Problem 5 — Tkinter Graphical User Interface

Interface Components

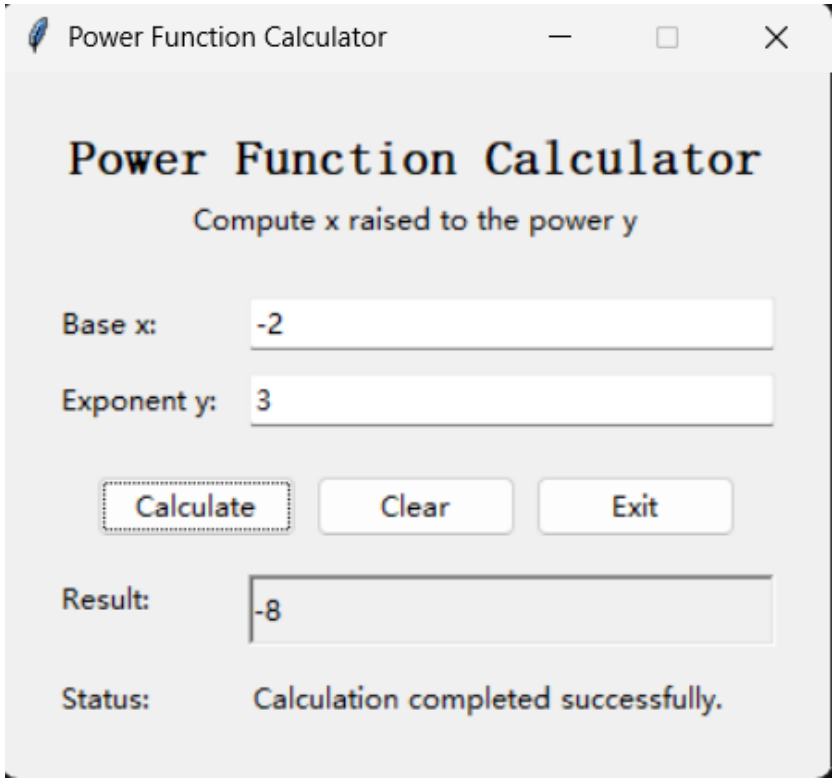
- Labelled fields for base x and exponent y
- Calculate, Clear, and Exit buttons
- Separate result and status areas
- Enter key triggers calculation

Button Behaviour

- **Calculate**: validates and computes
- **Clear**: resets all fields
- **Exit**: closes the application

Usability Goal

The interface remains open after invalid input so the user can correct the values and try again.



Successful GUI calculation: $(-2)^3 = -8$

Implementation

The class `PowerCalculatorApp` manages the widgets, event handling, result formatting, and status messages.

Problem 5 — Exception and Error Handling

Custom Exceptions

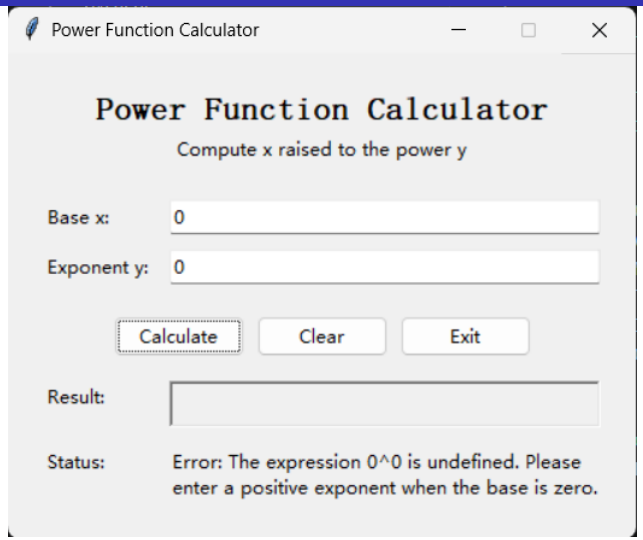
- `InvalidInputError`
- `UnsupportedDomainError`
- `NumericRangeError`
- `ConvergenceError`

GUI Recovery

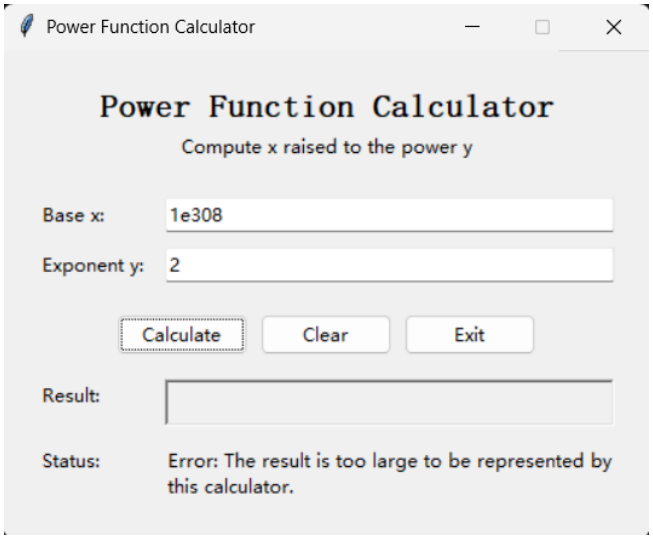
- Catches expected calculator errors
- Clears the result field
- Displays a plain-language message
- Keeps the application open

User Feedback

Messages explain the error and how the input should be corrected.



Domain validation: 0^0 is rejected with guidance.



Numeric-range validation: overflow is detected and reported.

Problem 5 — Verification Results

Base x	Exponent y	Observed Result	Status
4	0.5	Result: 2	Passed
2	-10	Result: 0.0009765625	Passed
-2	3	Result: -8	Passed
0	3	Result: 0	Passed
0	0	Rejected: 0^0 is undefined	Passed
-2	0.5	Rejected: integer exponent required	Passed
abc	2	Rejected: numeric base required	Passed
10^{308}	2	Overflow detected	Passed
10^{-308}	2	Severe underflow detected	Passed

Verification Summary

All representative valid inputs produced the expected result. Invalid domain, input, overflow, and underflow cases displayed helpful messages without terminating the GUI.

Problem 6 — GitHub Repository and README

Public Repository

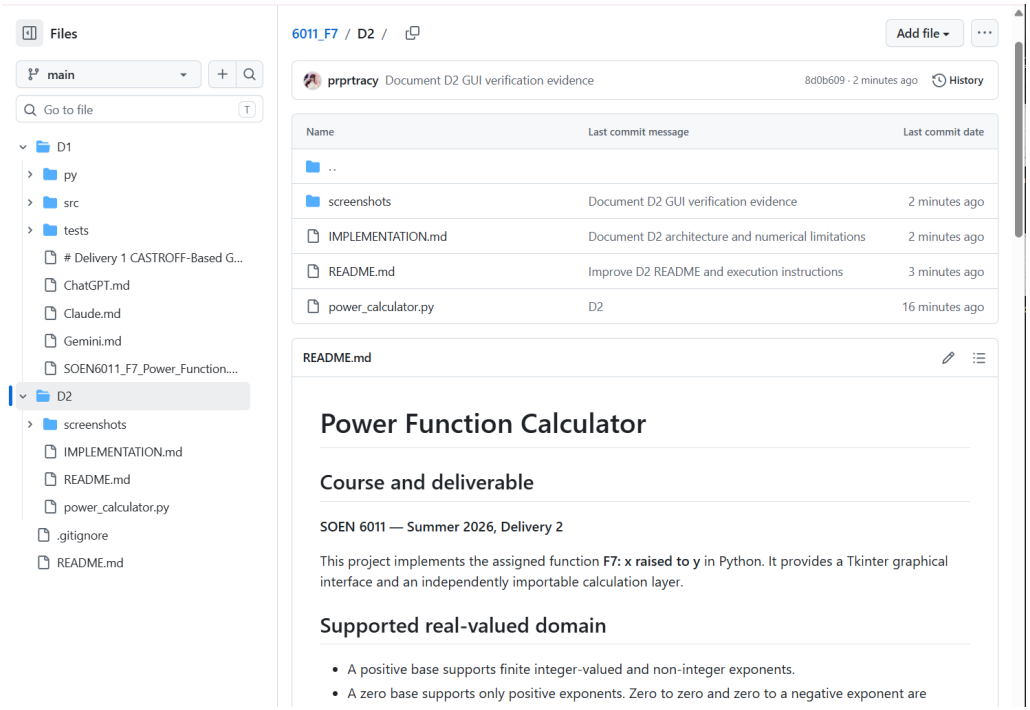
- Hosted using GitHub
- Publicly accessible
- D1 and D2 are stored separately
- Source code, screenshots, and README are included

README Contents

- Supported mathematical domain
- From-scratch algorithms
- Execution instructions
- Exception handling
- Test cases and limitations

Repository Address

`https://github.com/prprtracy/6011_F7.git`



Public repository containing the D2 source code, screenshots, and README.

High-Quality Commit Messages

- `Implement from-scratch hybrid power evaluation`
- `Add domain validation and custom exceptions`
- `Document D2 algorithms, tests, and limitations`

Problem 7 — Updated Functional and Input Requirements

ID	Updated Requirement	Change
FR-001	The system shall compute x^y within the supported real-valued domain using from-scratch numerical algorithms.	Modified
IR-001	The system shall accept the base x and exponent y through separately labelled input fields.	Modified
IR-002	The system shall accept only finite numeric values for x and y.	Retained
IR-003	The system shall determine whether y is integer-valued without using prohibited numerical-library functions.	Modified
IR-004	The system shall allow the user to correct invalid input and perform another calculation without restarting the application.	Modified
CR-001	The system shall not use built-in or library power, logarithm, or exponential operations to evaluate x^y.	New

D1-to-D2 Modification

The supported mathematical domain remains consistent with D1. The principal changes are the Tkinter input mechanism, custom numerical validation, and the from-scratch implementation of the subordinate logarithm and exponential functions.

Problem 7 — Updated Output, Error, and Usability Requirements

ID	Updated Requirement	Change
OR-001	The system shall display the computed result in a clearly labelled result area.	Modified
ER-001	The system shall reject $x = 0$ when $y \leq 0$ and display a helpful error message.	Retained
ER-002	The system shall reject $x < 0$ when y is not integer-valued and explain the supported domain.	Retained
ER-003	The system shall detect overflow, severe underflow, and convergence failure and display a helpful error message.	Modified
ER-004	The system shall recover from invalid input without closing the GUI.	Modified
UR-001	The system shall use plain-language labels, status messages, and error messages.	Retained
UR-002	The system shall provide a Tkinter graphical user interface for input and output.	Modified
UR-003	The system shall provide Calculate, Clear, and Exit controls and support calculation through the Enter key.	Modified
AR-001	For predefined independently verified test cases, the computed result shall have an absolute error of at most 10^{-9} when the expected result is zero, and a relative error of at most 10^{-9} otherwise.	Retained

Traceability

Every updated requirement is traceable to a GUI component, validation rule, calculation function, exception class, or verification case implemented in Problem 5.

GAI Use in Delivery 2

Part	Prompt Type	GAI Contribution	Student Review
P5	Code generation and refinement	Drafted the from-scratch hybrid algorithm, Tkinter GUI, validation logic, custom exceptions, and numerical safeguards.	Inspected the source code, checked the supported domain, searched for prohibited operations, and manually tested the GUI.
P6	Repository organization and documentation	Suggested the repository structure, README updates, implementation notes, screenshot documentation, and commit messages.	Reviewed all changed files, confirmed meaningful commits, and verified the public GitHub repository.
P7	Requirements revision	Drafted updated ISO/IEC/IEEE 29148-style requirements based on the D2 implementation.	Checked each requirement against the code and classified it as retained, modified, or new.

Use of GAI

GAI was used for drafting, comparison, organization, and refinement. All outputs were reviewed against the actual source code, repository, screenshots, and D2 project requirements.

CASTROFF Prompt and Student Responsibility

CASTROFF-Based Prompt Structure

Context: SOEN 6011 Delivery 2 for function x^y, extending the D1 hybrid algorithm.

Audience: Course evaluator and engineering-student persona.

Scope: Problems 5–7 only.

Task: Implement, verify, document, and revise the requirements.

Restrictions: No built-in mathematical power, logarithm, or exponential operations.

Output: Working code, repository evidence, documentation, and updated requirements.

Final Review: Compare all claims with the project description and actual source code.

Student Responsibility

GAI output was treated as a draft rather than an authoritative source. The final code, documentation, screenshots, repository history, requirements, and presentation content were reviewed and verified by the student.

Evidence Used for Review

The review was based on manual GUI tests, source-code inspection, GitHub history, repository documentation, and traceability between Problem 5 and the updated requirements in Problem 7.

References

Concordia University,
SOEN 6011 Project Description,
Summer 2026.

ISO/IEC/IEEE,
ISO/IEC/IEEE 29148:2018 Systems and Software Engineering — Life Cycle Processes — Requirements Engineering,
2018.

H. Tavakoli,
"The CASTROFF Framework: A Professional Standard for Prompt Engineering,"
in *Prompt Engineering for Everyone*, Apress, 2026.

Python Software Foundation,
Python 3 Documentation: tkinter — Python Interface to Tcl/Tk.

TkDocs,
Tkinter Tutorial.

OpenStax,
Precalculus 2e,
sections on exponential and logarithmic functions.

Y. Huang,
SOEN 6011 F7 GitHub Repository,
https://github.com/prprtracy/6011_F7.git.

Thank You

Questions are welcome.

Yichen Huang
SOEN 6011 — Delivery 2
Function F7: x^y